

Two parts. **Part A** is the prescribed Binomial Min Heap implementation. **Part B** is a creative implementation task: design an automatic visualizer for the *actual trees* created by Part A.

Use OOP concepts throughout the assignment. Your code should be modular, reusable, and easy to explain during evaluation.

1 Part A: Binomial Heap Implementation

Implement a **Binomial Min Heap**. Maintain two heaps, H_1 and H_2 , both initially empty. Every tree must be heap-ordered, and there must be at most one Binomial Tree of each order k .

For $U \ h1 \ h2$, store $\text{Union}(H_{h1}, H_{h2})$ in H_{h1} and make H_{h2} empty. Keys are integers and are unique across the two heaps at any instant.

1.1 Required Commands

Operation	Command	Meaning	Time
Insert	I h x	Insert x into H_h .	$O(\log n)$
Find Min	F h	Return the minimum key of H_h .	$O(\log n)$
Extract Min	E h	Return and remove the minimum key.	$O(\log n)$
Decrease Key	D h x y	Decrease key x to y .	$O(\log n)^*$
Remove Key	R h x	Delete key x .	$O(\log n)^*$
Union	U h1 h2	$H_{h1} \leftarrow \text{Union}(H_{h1}, H_{h2})$; empty H_{h2} .	$O(\log n)$
Print	P h	Print the heap in the fixed format below.	–

*For Decrease Key and Remove Key, the stated bound applies after the node containing x has been located, matching the handle/pointer assumption used in class.

Decrease Key: exchange *key values* with parents as needed; do not physically swap tree nodes.

Remove Key: decrease the key to a value **strictly smaller than** -10^9 (the minimum valid input key), then perform Extract Min. Do not use -10^9 itself as the sentinel, because -10^9 may already be present in the heap.

1.2 Union Convention

Use the following rules so that provided output is reproducible:

1. Keep root lists in increasing order of degree.
2. Consolidate left to right. With three consecutive roots of the same degree, do not link the first two; advance to the second root. Otherwise link equal-degree neighbors.
3. The smaller root key becomes the parent; the other root becomes its **first child**.
4. For Insert, create a one-node heap and perform Union with the current heap, as discussed in class.

1.3 Standard Print Format

The P h command is a fixed output used for automatic checking. It is **not** the creative visualizer of Part B.

- Print heap size.

- Print trees in order $B_0, B_1, B_2, \dots$
- Print each tree level by level.
- Within one level, print keys in increasing numerical order.

Non-empty heap example:

```
Printing Binomial Heap H1
Heap size: 5
Binomial Tree, B0
Level 0: 30
Binomial Tree, B2
Level 0: 5
Level 1: 7 20
Level 2: 12
```

Empty heap:

```
Printing Binomial Heap H1
Heap size: 0
Heap H1 is empty.
```

For `F h` and `E h`, print exactly:

```
Find Min returned: x
Extract Min returned: x
```

Formatting is case-sensitive. Do not print extra prompts, debug messages, or blank lines in normal Part A output.

What Part A Print checks: the provided output is used to check heap size, the set of B_k trees, and the level of each key. Because keys within a level are printed in sorted order, complete parent–child relationships are **not** determined by `P h`; those relationships are shown and manually evaluated in Part B.

1.4 Input, Output, and Evaluation

Input must be read from `input.txt`.

Every required output line must be written to `output.txt` and also printed to the console. The two outputs must be identical.

Insert, Decrease Key, Remove Key, and Union produce no normal Part A output. Find Min, Extract Min, and Print produce output exactly as specified.

During evaluation, you must be able to compare your `output.txt` with the provided output file using Diffchecker or an equivalent line-by-line comparison tool.

1.5 Assumptions and Restrictions

- At most 10^5 commands; valid input keys are in $[-10^9, 10^9]$.
- `I h x`: x does not exist in either heap.
- `D h x y`: x exists in H_h , $y < x$, and y does not exist in either heap.
- `R h x`: x exists in H_h .
- Find Min / Extract Min will not be called on an empty heap.
- Union heap identifiers are always different; all commands are otherwise valid.
- **Do not rebuild a heap from all stored keys after an operation.**
- **Do not replace the Binomial Heap with a built-in heap/priority queue.** Ordinary containers may be used only as supporting storage, e.g., for printing, lookup, or visualization.

1.6 Sample Input / Output

input.txt

```
I 1 20
I 1 5
I 1 30
I 2 12
I 2 7
P 1
P 2
U 1 2
P 1
D 1 30 3
F 1
E 1
P 1
```

output.txt and console

```
Printing Binomial Heap H1
Heap size: 3
Binomial Tree, B0
Level 0: 30
Binomial Tree, B1
Level 0: 5
Level 1: 20
Printing Binomial Heap H2
Heap size: 2
Binomial Tree, B1
Level 0: 7
Level 1: 12
Printing Binomial Heap H1
Heap size: 5
Binomial Tree, B0
Level 0: 30
Binomial Tree, B2
Level 0: 5
Level 1: 7 20
Level 2: 12
Find Min returned: 3
Extract Min returned: 3
Printing Binomial Heap H1
Heap size: 4
Binomial Tree, B2
Level 0: 5
Level 1: 7 20
Level 2: 12
```

2 Part B: Creative Implementation – Binomial Heap Visualizer

You are NOT designing new Binomial Trees. Your Part A code determines every node and parent–child relationship. In Part B, your job is to design **how those actual trees are automatically visualized.**

There is no prescribed visual format and no sample visualization. The layout and presentation are part of your implementation design.

2.1 Your Visualizer Must

1. **Visualize a heap:** show every B_k separately, identify its order and root, and make actual parent–child relationships clear. A level-order list alone is not enough.
2. **Visualize Union:** show the two heaps before Union, every link $B_k + B_k \rightarrow B_{k+1}$ in order, and the final heap.
3. **Add one useful feature of your own design** that communicates extra structural/operational information. A purely decorative change does not count.

2.2 Implementation Rules for Part B

- Reuse the **same heap and node structures** from Part A; do not create a separate fake heap for visualization.
- Temporary layout data (node pointers, coordinates, widths, etc.) is allowed.
- Text/ASCII/Unicode, terminal output, coordinates, or a GUI may be used. A GUI is not required.
- You must compute the layout yourself. Automatic tree/graph layout tools such as Graphviz are not allowed. Basic drawing libraries are allowed if your code computes all positions.
- The evaluator will construct heaps only through valid Part A commands. For Part B evaluation, there will be at most **31 elements** in total and no displayed tree larger than B_4 .

- Document how to invoke the heap and Union visualizations in `README.txt`.

3 Mark Distribution (Total: 100)

Component	Marks
Part A: Binomial Heap Implementation	60
Part B: Creative Binomial Heap Visualizer	30
Evaluation / Viva	10
Total	100

4 Submission and Evaluation

- Use **OOP concepts** and write readable, reusable, well-structured code.
- Part A and Part B must use the same Binomial Heap implementation.
- Include `README.txt` with compile/run instructions, file-I/O instructions, Part B invocation, and a brief description of your own visualization feature.
- During evaluation, be prepared to run the provided `input.txt`, generate `output.txt`, show the console output, and demonstrate the diff comparison.
- Submit source files and `README.txt` inside a directory named with your 7-digit student ID. Compress it as `.zip` and submit through Moodle.